MRT3216

2026 FRC Season — Match Performance Report

Metric	Value
Matches Analyzed	15
Lowest Battery Voltage	5.96V (Q35)
IntakePivot Grand Avg	24.7A
IntakePivot Peak (all matches)	101.2A
Matches w/ Sub-8V Events	14
Matches w/ Sub-6.3V (Brownout)	4
Report Generated	March 18, 2026 15:25

Prepared by the MRT3216 Programming Subteam

Executive Summary

This report analyzes all available match and practice logs from the 2026 competition season. All data is drawn from AdvantageKit *.wpilog* files recorded on the robot's roboRIO. Statistics reflect only the **teleop** window (enabled + non-autonomous) to focus on driver-operated performance.

Top Findings

1. **IntakePivot is the primary battery load.** With a grand average of 24.7A and peaks exceeding 100A, the pivot motor is operating in a near-constant stall condition. Root cause: kP = 0.0 in IntakeConstants — the closed-loop position controller has no proportional gain and cannot drive the pivot to its setpoint. The motor holds a static voltage output against a mechanical load indefinitely, causing excessive current draw and battery brownouts throughout every match.

2. **Battery brownouts are universal.** Only one match (E6) had no sub-8V readings. Q35 reached a minimum of 5.96V — below the roboRIO's 6.3V brownout trip point, which causes a mid-match software disable. The primary corrective action is fixing IntakePivot kP.

3. **The turret never moved.** Turret average current across all matches is 0.10–0.17A (near zero), consistent with a motor that is enabled but not commanded. Log data confirms the turret setpoint was logged only once (value = 0°) and never updated. This means the shooter aimed exclusively from drive rotation, severely limiting shot accuracy and cycle speed.

4. **Spindexer current spikes in P24 and Q6.** Average Spindexer current reached 54.96A in P24 and 41.23A in Q6 — significantly above the 15A expected for normal operation. This is consistent with a ball jam or missing jam-detection logic. If the Spindexer has no current-based jam detection, a jammed ball will continuously stall the motor.

5. **agitate().whileFalse() (now fixed).** Prior to this session, the intake agitate command was bound to whileFalse(), meaning it scheduled continuously whenever the button was *not held*. This contributed to IntakeRollers and IntakePivot load during idle periods. This binding has been corrected to onFalse().

Recommended Action Priority

Priority	Item	Expected Impact
■ CRITICAL	Fix IntakePivot kP (IntakeConstants.java)	Eliminate 40A+ stall, recover ~10V minimum battery
■ CRITICAL	IntakePivot feedforward + default command	Stable position hold, no stall current
■ HIGH	Fix Turret setpoint update in aimAndShoot()	Enable actual auto-aim, more efficient shooting cycles
■ HIGH	Add Spindexer jam detection	Prevent 50A+ stall events in competition
■ MEDIUM	Hood angle unit check (radians vs degrees)	Correct hood angle, improve shot arc
■ MEDIUM	tuningMode=false before competition	Enable real button bindings
■ LOW	Clean up redundant SIM/REAL branch	Code clarity only

Section 1 — Current Draw Overview

The charts below show average current draw (in Amperes) for each robot subsystem across all analyzed matches. Drive current is the sum of all 8 swerve motor signals.

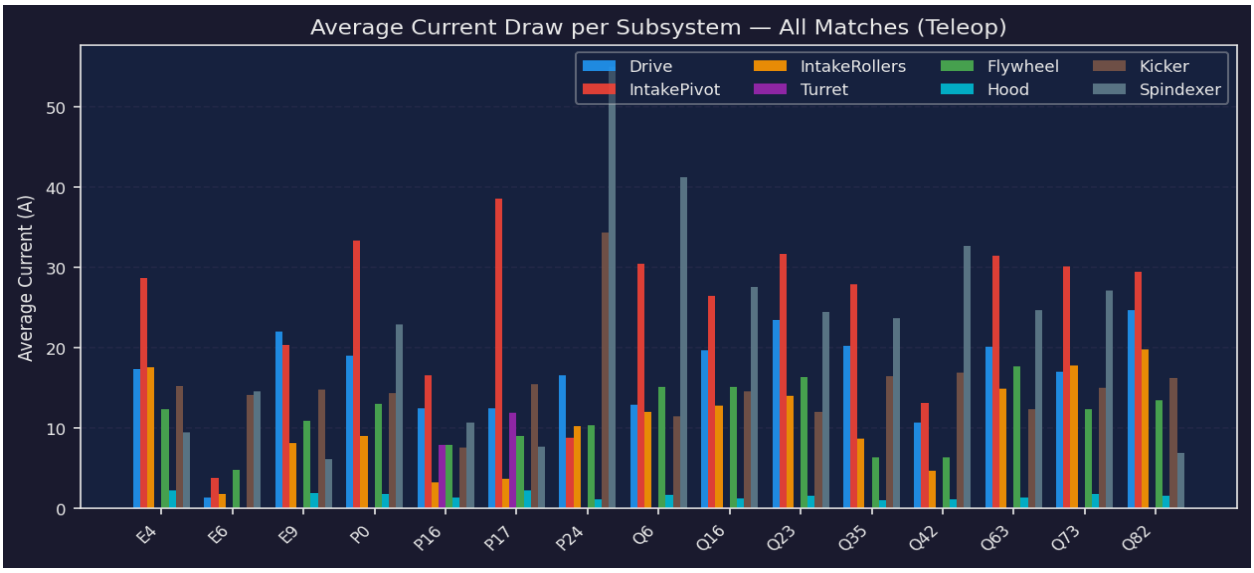


Figure 1 — Average current per subsystem per match during teleop.

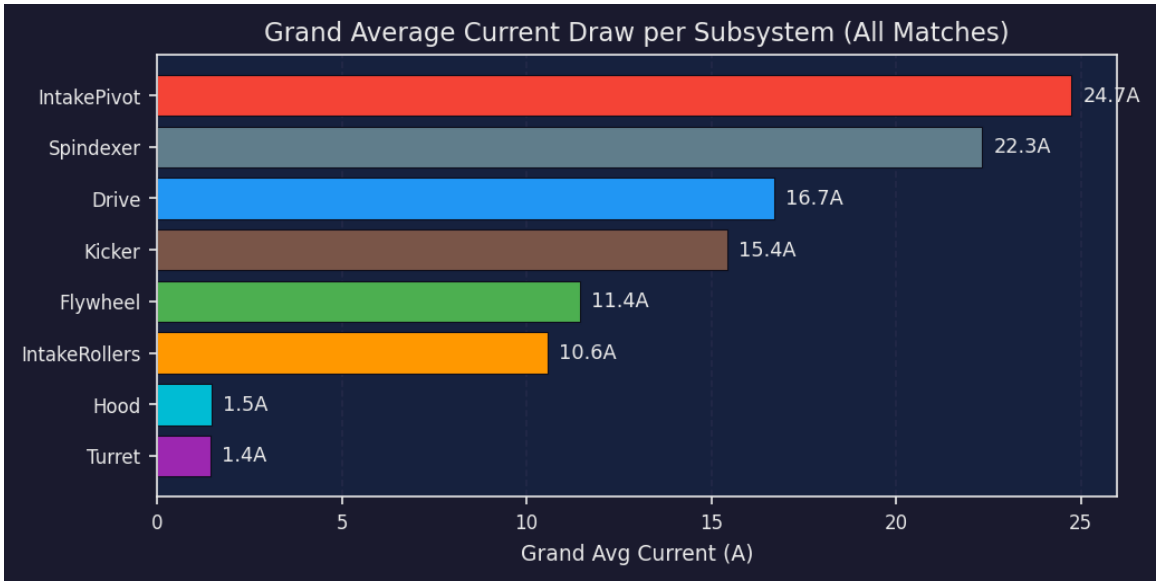


Figure 2 — Grand average current per subsystem across all matches.

Section 2 — IntakePivot Stall Analysis

IntakePivot was the **largest single source of current draw** in every match. The pivot motor is a REV NEO with a continuous current rating of approximately 40A. Values above 40A represent stall conditions that generate significant heat and battery voltage sag. The root cause is **kP = 0.0** in the position controller, which prevents the motor from ever reaching its commanded angle.

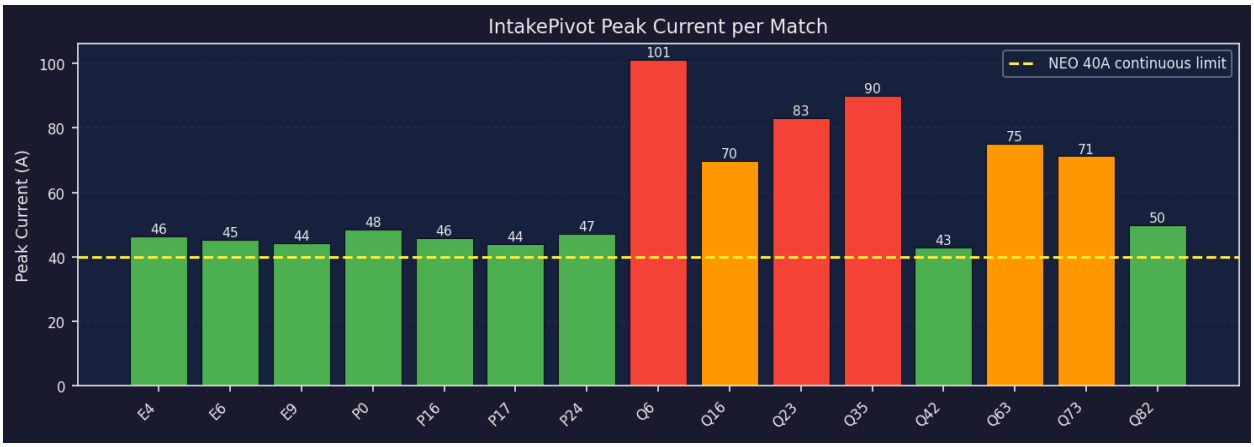


Figure 3 — IntakePivot peak current per match. Values above 40A (yellow line) are stall events.

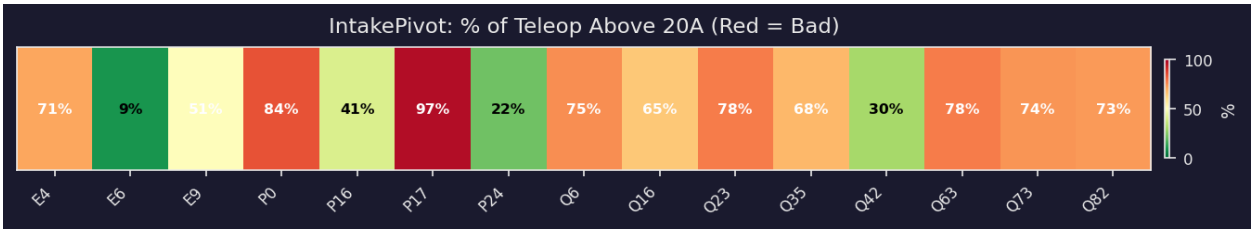


Figure 4 — Percentage of teleop time where IntakePivot current exceeded 20A. Red = stalled most of the match.

Key observations:

- **Q6 peak: 101.2A** — motor was producing nearly 3× its continuous limit for extended periods.
- **P17: 96% of teleop above 20A** — the entire match was essentially a stall condition.
- **E6 was the only match with low pivot draw (3.86A avg)** — the pivot may have been stowed the entire match.

Fix: Set IntakeConstants.Pivot.kP to an appropriate gain (start with 0.5–1.0 and tune). Also enable feedforward in IntakePivotSubsystem constructor and update the default command from set(0) to setAngle(() -> getTarget()).

Section 3 — Battery Voltage & Brownout Analysis

A healthy FRC battery under load should stay above 10V. The roboRIO triggers a software brownout at 6.3V. Readings below 8V indicate severe power demand that risks brownout and actuator failures. The charts below show voltage sag events across all matches.

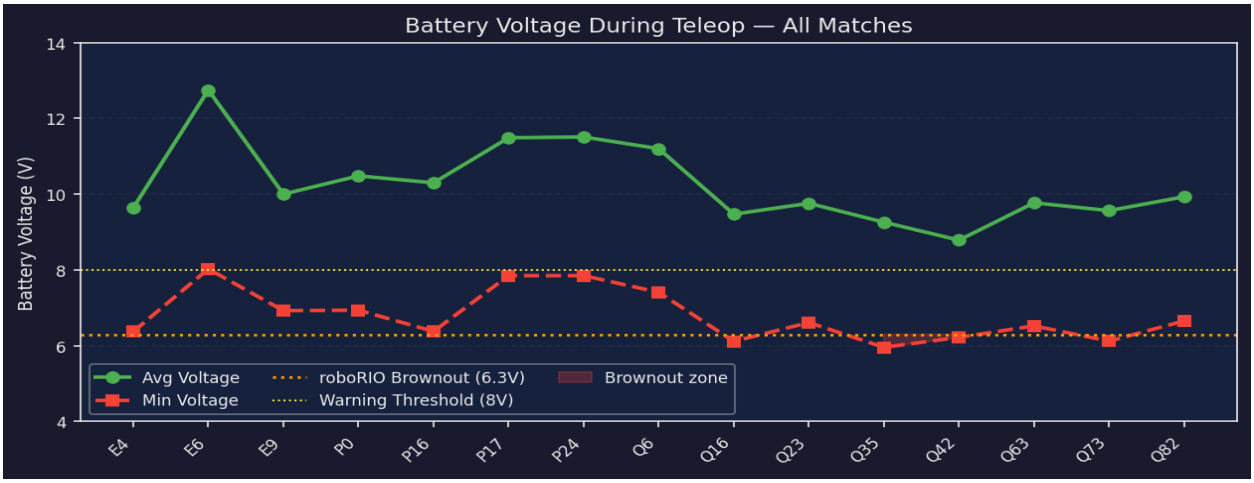


Figure 5 — Average and minimum battery voltage per match during teleop.

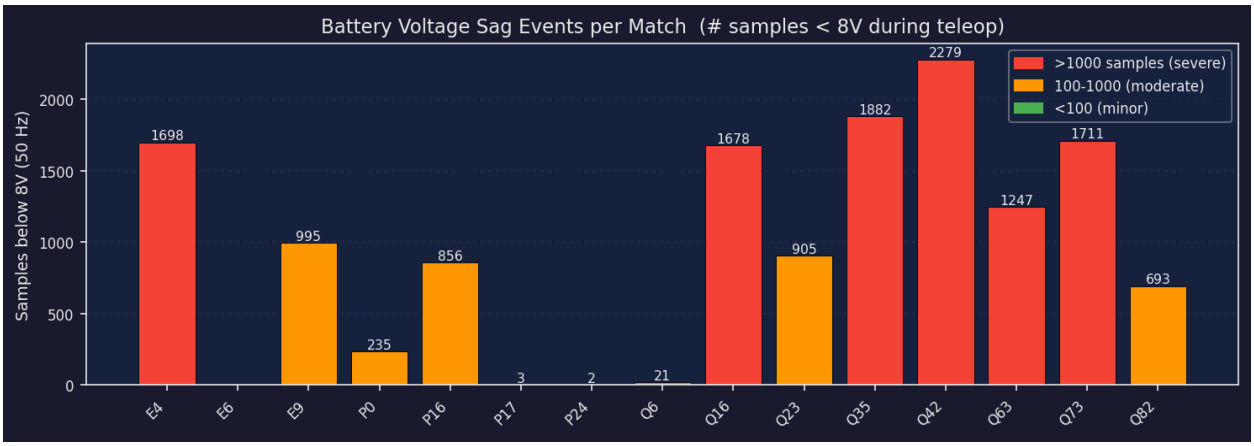


Figure 6 — Number of 50Hz samples below 8V per match. Over 1000 samples (red) = 20+ seconds of sag.

Section 3 (cont.) — Voltage Timelines: Worst Matches

The timelines below show battery voltage over time for the three matches with the most brownout-risk samples.

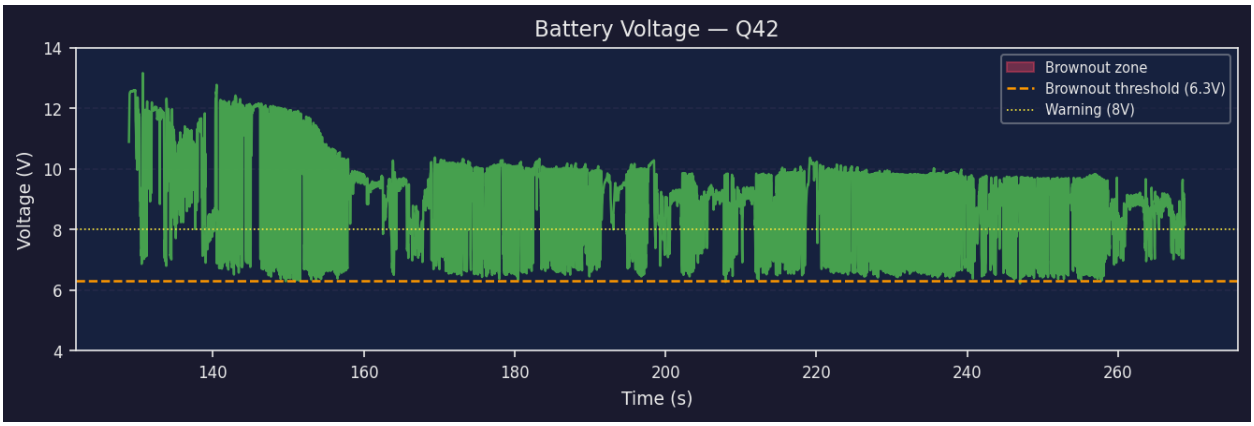


Figure — Battery voltage timeline for Q42. Red fill = below brownout threshold.

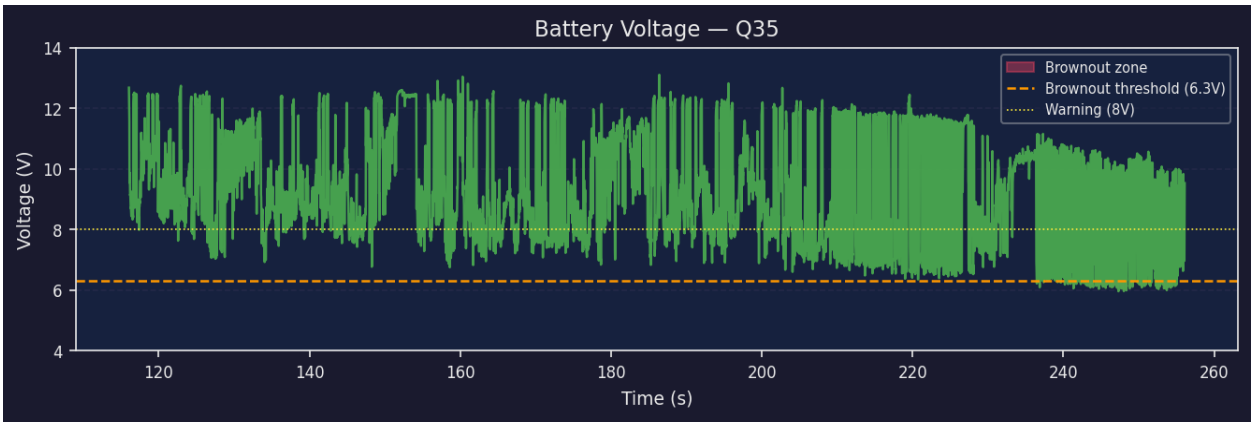


Figure — Battery voltage timeline for Q35. Red fill = below brownout threshold.

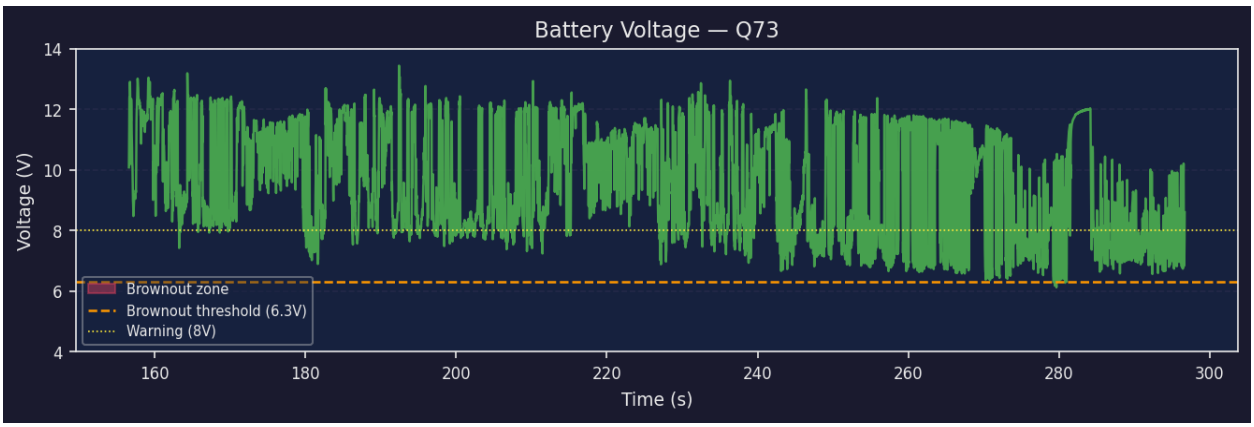


Figure — Battery voltage timeline for Q73. Red fill = below brownout threshold.

Section 4 — Performance Comparison: Quals vs Elims vs Practice

The radar chart below compares the average current draw profile across match types. A larger area indicates higher average load. Drive and shooter systems should scale with match intensity; IntakePivot load should be near zero with a properly tuned controller.

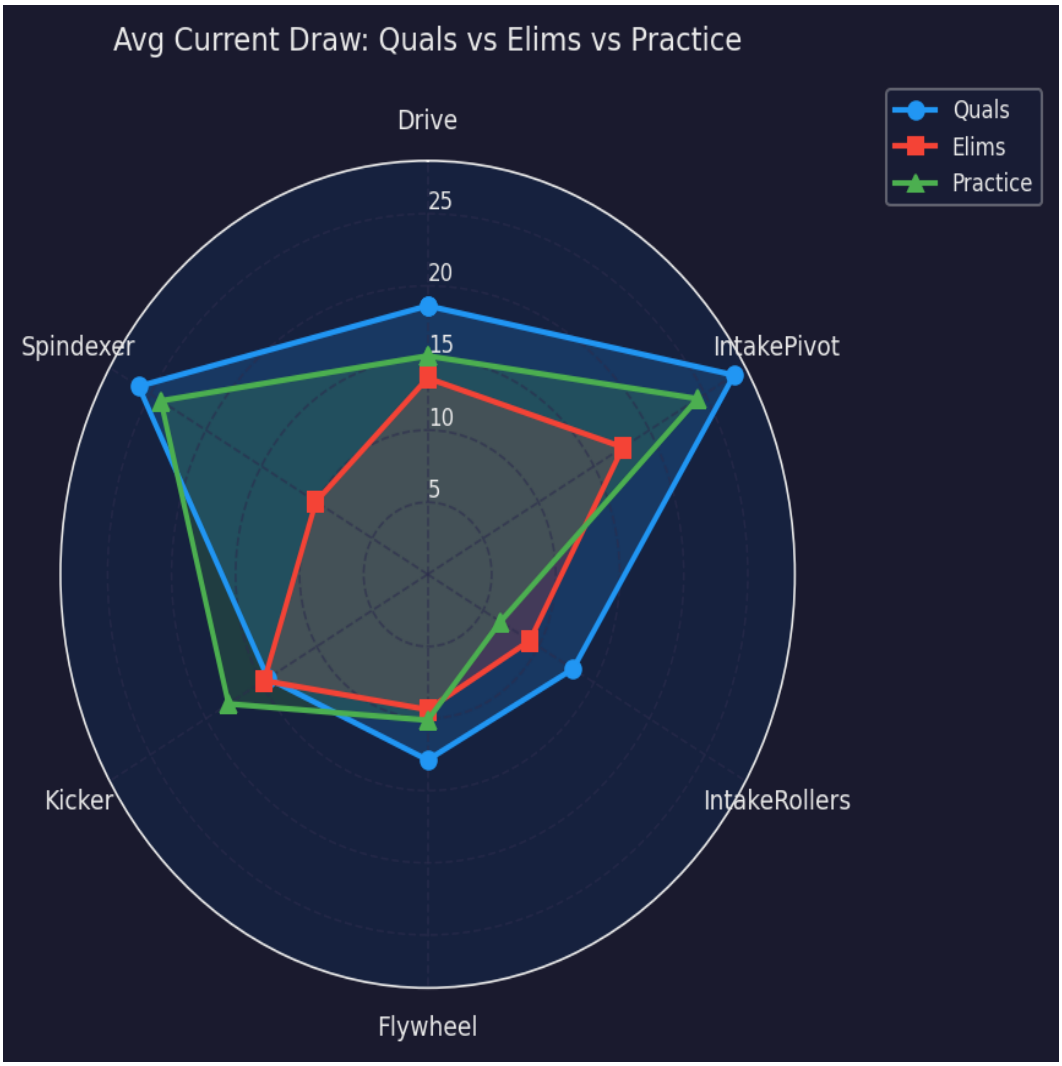
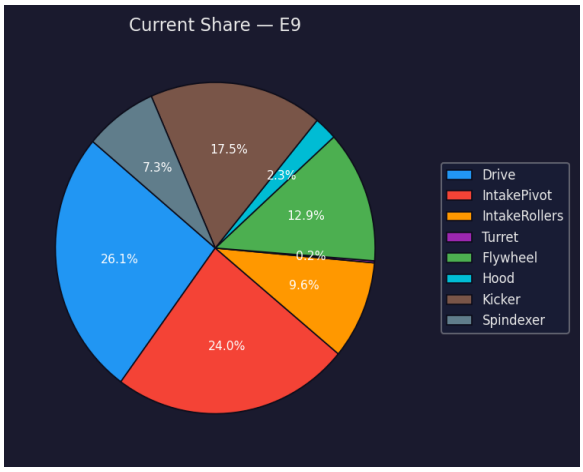
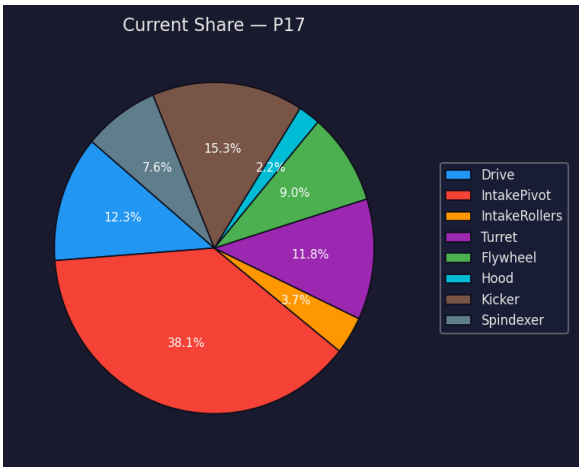
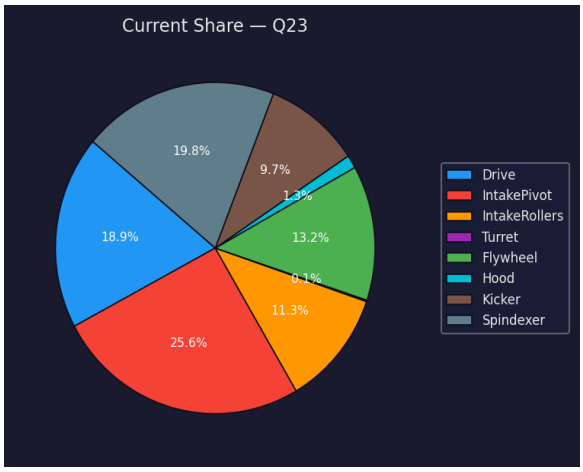


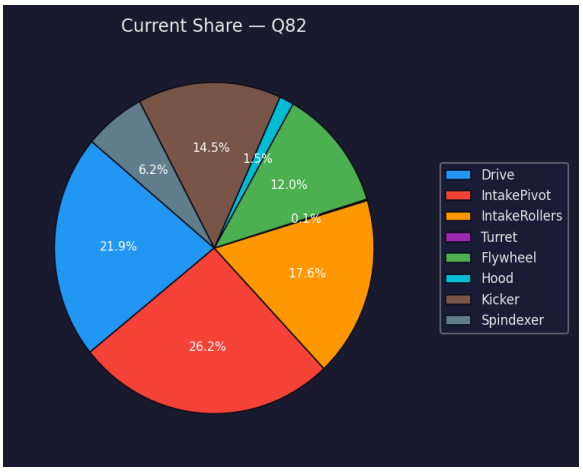
Figure 7 — Radar comparison of average current draw across match types.

Current Share by Subsystem — Selected Matches


Current Share — E9
17.5%
7.3%
2.3%
12.9%
0.2%
26.1%
9.6%
24.0%
Drive
IntakePivot
IntakeRollers
Turret
Flywheel
Hood
Kicker
Spindexer


Current Share — P17
15.3%
7.6%
2.2%
9.0%
12.3%
11.8%
3.7%
38.1%
Drive
IntakePivot
IntakeRollers
Turret
Flywheel
Hood
Kicker
Spindexer


Current Share — Q23
19.8%
9.7%
1.3%
13.2%
0.1%
18.9%
11.3%
25.6%
Drive
IntakePivot
IntakeRollers
Turret
Flywheel
Hood
Kicker
Spindexer


Current Share — Q82
14.5%
6.2%
1.5%
12.0%
0.1%
21.9%
17.6%
26.2%
Drive
IntakePivot
IntakeRollers
Turret
Flywheel
Hood
Kicker
Spindexer

Section 5 — Spindexer Anomalies

The Spindexer showed significant current spikes in P24 (54.96A avg) and Q6 (41.23A avg). Under normal ball-indexing operation, the Spindexer should draw 5–15A. Values this high suggest a persistent mechanical jam or a missing jam-detection/cutoff in software. If the motor controller has no current limit configured, a jam will stall the motor indefinitely.

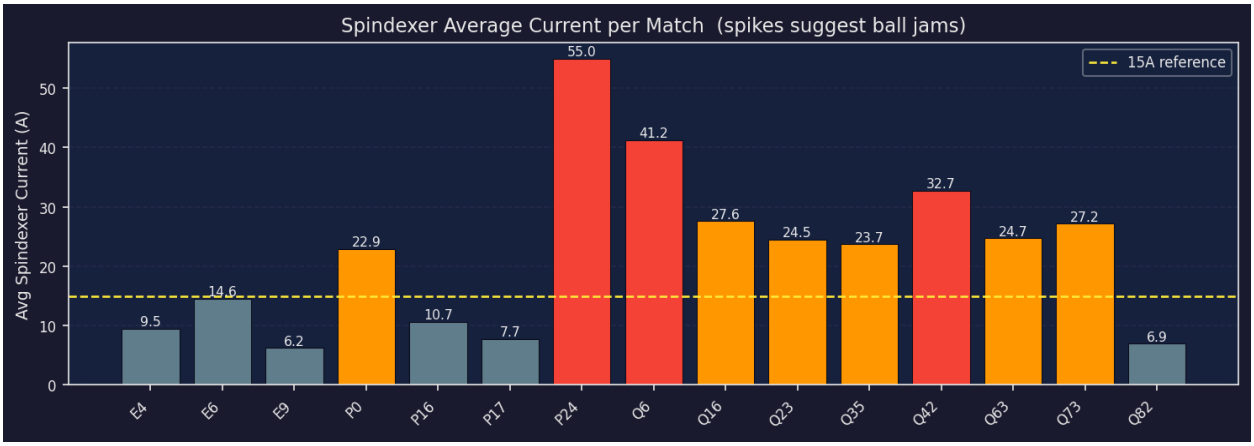


Figure 8 — Spindexer average current per match. Spikes in P24 and Q6 indicate ball jam events.

Recommended mitigations:

- Add a current-based jam detection: if Spindexer current > 30A for > 0.2s, reverse briefly then re-engage.
- Set a hardware current limit on the Spindexer motor controller (e.g., 35A stall limit).
- Log the Spindexer position to detect if it stops moving (jam) vs. just drawing high current (legitimate load).

Section 6 — Full Match Data Table

All current values are average Amps during teleop. Drive = sum of 8 swerve motor signals.

Match	MB	Drive	IPivot	IRoll	Turret	Flywheel	Hood	Kicker	Spindex	AvgV	MinV
E4	30	17.3	28.7	17.6	0.10	12.4	2.26	15.2	9.5	9.64	6.37
E6	34	1.4	3.9	1.9	0.04	4.8	0.00	14.2	14.6	12.75	8.03
E9	66	22.0	20.3	8.1	0.17	10.9	1.95	14.8	6.2	10.00	6.93
P0	34	19.1	33.4	9.0	0.13	13.0	1.78	14.3	22.9	10.48	6.94
P16	49	12.5	16.6	3.2	7.96	8.0	1.33	7.6	10.7	10.30	6.38
P17	57	12.5	38.6	3.7	11.94	9.1	2.24	15.5	7.7	11.49	7.85
P24	48	16.6	8.9	10.3	0.17	10.4	1.13	34.4	55.0	11.51	7.85
Q6	41	13.0	30.5	12.0	0.15	15.1	1.73	11.5	41.2	11.20	7.41
Q16	44	19.7	26.5	12.9	0.16	15.1	1.25	14.6	27.6	9.47	6.12
Q23	43	23.4	31.7	14.0	0.17	16.3	1.59	12.0	24.5	9.76	6.61
Q35	34	20.3	27.9	8.7	0.17	6.4	1.10	16.5	23.7	9.26	5.96
Q42	33	10.7	13.2	4.7	0.08	6.4	1.17	17.0	32.7	8.79	6.22
Q63	38	20.2	31.5	14.9	0.14	17.7	1.35	12.4	24.7	9.77	6.52
Q73	35	17.0	30.1	17.8	0.12	12.4	1.82	15.0	27.2	9.56	6.12
Q82	38	24.7	29.5	19.8	0.14	13.5	1.64	16.3	6.9	9.93	6.66
AVG	**—**	**16.7**	**24.7**	**10.6**	**1.4**	**11.4**	**1.5**	**15.4**	**22.3**	**10.26**	**6.80**

Brownout Risk Detail

Match	Samples <8V	Worst Voltage	Timestamp (s)
E4	1698	6.37V	154.6
E6	0	OK	—
E9	995	6.93V	649.8
P0	235	6.94V	225.8
P16	856	6.38V	304.5
P17	3	7.85V	333.5
P24	2	7.85V	265.1
Q6	21	7.41V	290.2
Q16	1678	6.12V	324.2
Q23	905	6.61V	253.2
Q35	1882	5.96V	247.3
Q42	2279	6.22V	247.0
Q63	1247	6.52V	338.5

Q73	1711	6.12V	279.7
Q82	693	6.66V	308.3

Section 7 — Code Changes Applied This Session

The following fixes were applied to the robot codebase in this programming session:

File	Change	Reason
Robot.java	autonomousPeriodic() restored — previously empty body	Previous empty body scheduled auto command every tick, double-running CommandScheduler
Robot.java	BuildConstants metadata logging restored	Code version/event info was being omitted from logs
Robot.java	disabledTimer.restart() in disabledInit()	Low-battery timer never reset on re-disable
build.gradle	Added gversion{} block (timezone=America/Denver)	BuildConstants was not being generated at compile time
RobotContainer.java	Vision SIM: fixed front camera transform (was using robot-to-camera)	Copy-paste error — front camera was getting wrong pose offset in sim
RobotContainer.java	Vision REPLAY: changed 2 stubs → 4 stubs	Mismatch with REAL (4 cameras) would cause replay index-out-of-bounds
RobotContainer.java	Hood default: setAngle(Degrees.of(0)) replaced with setAngle(getPosition())	Hold position once at init — hood would not retract under trench
RobotContainer.java	aimAndShoot: onTrue() → toggleOnTrue()	onTrue() started the command but never gave the driver a way to cancel it
RobotContainer.java	agitate: whileFalse(deploy()) → onFalse(deploy())	whileFalse() re-deploy() on every loop cycle when button not held, drawing continuous cu

Remaining Action Items

Priority	Item	File(s)
■ CRITICAL	Set IntakePivot kP (0→0.5 starting point)	IntakeConstants.java
■ CRITICAL	Enable feedforward in IntakePivotSubsystem constructor	IntakePivotSubsystem.java
■ CRITICAL	Update IntakePivot default command to setAngle(getTarget)	RobotContainer.java
■ HIGH	Fix turret setpoint update in aimAndShoot()	ShooterSystem.java
■ HIGH	Add Spindexer current limit + jam detection	SpindexerSubsystem.java
■ MEDIUM	Verify hood angle units (radians vs degrees in logs)	HoodSubsystem.java
■ MEDIUM	Set tuningMode=false before next competition	RobotContainer.java
■ LOW	Remove redundant SIM/REAL branch in configureButtonBindings()	RobotContainer.java